Ave: Guiding Agentic GPU Optimization Using Data-Flow Invariants

Haohui Mai[♪,1] Xiaoyan Guo[4] Xiangyun Ding[♪,5] Daifeng Li[1] Qiuchu Yu[4]
Chenzhun Guo[4] Cong Wang[2] Jiacheng Zhao[4] Christos Kozyrakis[3] Binhang Yuan[1]
CausalFlow Inc.[♪] HKUST[1] Tsinghua University[2] Stanford University[3] UCAS[4] UC Riverside[5]

arXiv:2604.18616v2 [cs.DC] 14 Sep 2026

Abstract

LLM coding agents can generate correct GPU kernels, yet their performance still lags behind expert libraries. Achieving peak GPU throughput requires coordinating low-level transformations such as shared-memory staging, software pipelining, and instruction scheduling. Unit tests and performance profiles provide only sparse end-to-end feedback, leaving agents unable to localize violations of the global constraints that these transformations must preserve.

We present Ave, an agentic framework that uses *data-flow invariants* as compile-time guardrails for kernel optimization. These invariants specify relationships among data values that must hold throughout execution. Ave's tile-based Pythonic DSL exposes hardware instructions and compiler policies while abstracting complex memory layout encodings as tiles. Tag functions assign symbolic labels derived from selected logical coordinates and expressions. The compiler propagates these labels through data and control flow, and tag assertions enforce required relationships among them at use sites. The compiler checks these assertions with a tractable, flow-sensitive, path-insensitive analysis backed by an SMT solver, returning concrete assignments that witness abstract violations and guide repairs with no runtime overhead. An in-context reinforcement learning (ICRL) planner proposes optimizations from a curated knowledge base. A lowering agent implements them and instantiates their invariants.

We evaluate Ave on AMD MI300X across GEMM, flash attention, and MoE, which together account for up to 90% of GPU time in LLM inference. With GPT-5.6 Sol, its optimized kernels achieve 89–99% of the effective throughput of state-of-the-art hand-optimized libraries and improve geometric-mean throughput by 1.62–1176× over uncontaminated agentic baselines. On 200 KernelBench tasks, with GPT-5.6 Sol and in-context DSL examples, Ave produces valid kernels within three attempts for 100% of Level 1 and 88% of Level 2 problems. Ave is open source at https://github.com/causalflow-ai/avelang.

CCS Concepts: • **Software and its engineering** → **Compilers**; **Automated static analysis**; • **Computing methodologies** → *Artificial intelligence.*

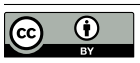
Keywords: GPU kernel optimization, LLM coding agents, data-flow invariants, static analysis

ACM Reference Format:
Haohui Mai, Xiaoyan Guo, Xiangyun Ding, Daifeng Li, Qiuchu Yu, Chenzhun Guo, Cong Wang, Jiacheng Zhao, Christos Kozyrakis, and Binhang Yuan. 2026. Ave: Guiding Agentic GPU Optimization Using Data-Flow Invariants. In *Symposium on Operating Systems Principles (SOSP '26), September 29–October 2, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830418.3843902

1 Introduction

Modern large language models (LLMs) [55] increasingly exhibit broad, human-level capabilities and may be approaching a tipping point for recursive self-improvement. GPU kernel optimization lets LLMs improve software governing their own execution efficiency [14, 16].

LLMs can now generate functionally correct kernels for diverse workloads [6, 9, 18, 25, 26, 32, 43, 56, 64, 70, 78], but their performance remains unsuitable for production. With GPT-5.6 Sol on AMD MI300X, existing agentic harness systems produce GEMM kernels that are 1.8–14.3× slower than hipBLASLt [4], AMD's state-of-the-art GEMM library. Closing this gap requires more than functional correctness: agents must coordinate optimizations across the software-hardware stack while preserving program semantics. For example, a high-performance GEMM implementation combines matrix cores [45], software pipelining [38], memory-hierarchy optimizations [42], and instruction scheduling [34], many of which require low-level assembly. Kernel-design agents [30, 72] combine expert knowledge bases with powerful models and iterate using unit-test outcomes and performance profiles, yet they struggle to apply performance-critical, low-level optimizations for two reasons. First, validating such transformations requires reasoning about non-trivial properties, including memory aliasing [60] and data races [7], at the assembly level. This reasoning consumes many tokens and expands the context, ultimately degrading



This work is licensed under a Creative Commons Attribution 4.0 International License.
SOSP '26, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/26/09
https://doi.org/10.1145/3830418.3843902

model performance [44]. Second, unit tests and performance profiling only provide sparse, end-to-end feedback. These signals reveal that a kernel is incorrect or slow but do not identify the responsible transformation or ordering.

One alternative is to trade optimization control for compiler automation by targeting a higher-level language such as Triton [66], which delegates many low-level decisions to a tensor compiler [14, 29, 46, 66, 69, 71, 80]. In our evaluation, the ROCm Triton baseline narrows the gap to approximately 2×. However, this baseline does not expose fine-grained control over instruction scheduling or wavefront specialization, preventing agents from closing the remaining gap.

In this paper, we present Ave, an agentic framework for automating complex, multilevel GPU kernel optimizations. Ave uses *data-flow invariants* to validate transformations and provide guardrails as LLMs apply them. These compile-time assertions specify relationships among data values that must hold throughout kernel execution, even across multiple layers of optimization. Data-flow invariants are an established concept in information-flow security [50, 73, 75]. Ave adapts them to GPU kernel optimization. For example, a GEMM kernel tiles and stages matrices **A** and **B** in shared memory before passing them to hardware matrix fused multiply-add (MFMA) instructions, which expect the operands in a particular swizzled layout. A data-flow invariant can assert that these instructions always receive correctly paired elements of **A** and **B**, regardless of the transformations applied. By making such requirements explicit, Ave gives the agent concrete correctness constraints for intrusive transformations such as software pipelining and data-layout reorganization.

Three challenges arise in realizing this approach. First, the invariant language must express global properties concisely in one place while producing concrete violation reports that guide repairs, such as the thread holding a mismatched element and the program point where the mismatch occurs. Second, enforcing invariants must incur zero runtime overhead. Optimized kernels saturate hardware units, tailor access patterns to the memory hierarchy, and depend on precise instruction schedules. The analysis must therefore operate entirely at compile time across tiling, staging, and layout transformations, without perturbing the generated code. Third, application-specific invariant templates must be instantiated automatically. The assignment of **A** and **B** tiles to wavefronts, their staging in shared memory, and their eventual consumption by MFMA instructions vary across GEMM kernels, so Ave must instantiate the corresponding invariants without per-kernel human intervention.

Ave combines three ideas: *tag functions*, *static analysis*, and an *in-context reinforcement learning (ICRL) agent*.

First, Ave introduces *tag functions and tag assertions*, a concise language for expressing data-flow invariants. It generates and optimizes kernels in a tile-based, Pythonic DSL inspired by CuTe [52] and TileLang [69]. Tag functions attach symbolic values, such as logical coordinates, to tensor elements and propagate them through control and data flow. Tag assertions require tags at a use site either to match or to differ. Conformity assertions encode alignment constraints, such as requiring paired matrix-multiplication operands to carry compatible tags. Non-conformity assertions encode separation constraints, such as ensuring that concurrent producers do not overwrite the same shared-memory region.

Second, Ave uses *static analysis* to enforce these invariants entirely at compile time and turn failures into actionable feedback for the LLM coding agent. The tags are purely symbolic and never materialized at runtime. The Ave compiler tracks their propagation using the abstract interpretation [17] over a layout algebra [52] and leverages an SMT solver [20] to solve the resulting integer constraints. To remain tractable for heavily optimized kernels, the analysis is path-insensitive. When an assertion fails, the compiler reports a concrete assignment witnessing the violating abstract state, including the thread ID, data element, and program point, thereby guiding the agent's next revision.

Third, Ave uses *ICRL* [26, 49] to adapt a planner that proposes optimizations and their corresponding invariant templates. The planner observes the kernel implementation, the compile-time invariant violations, and the runtime profiles, retrieves applicable optimizations from a persistent knowledge base, and proposes optimization instructions with their templates (Figure 1). A lowering agent binds the selected instructions to code and instantiates the templates as tag functions and assertions. Each invariant specifies a property that its transformation must preserve. Ave makes planner prompts learnable, uses invariant violations and runtime performance as reward signals, and applies text gradients [74] to improve subsequent optimization plans.

These three ideas reinforce one another. Tag functions alone provide only declarative specifications. Static analysis can check properties but cannot repair violations, and an ICRL agent alone lacks the structured feedback needed to navigate a vast optimization space. Together, they form a closed loop: tag functions express correctness properties, static analysis checks them at compile time, and concrete assignments witnessing abstract violations provide dense feedback. This feedback lets the agent reason about abstract correctness properties instead of repairing implementations through trial and error. Ave thereby applies complex, multilevel optimizations more reliably while reducing the effort required to preserve correctness. Discovering new algorithmic optimizations [30] and fully verifying implementation correctness [7, 11] remain outside its scope.

We evaluate Ave on three production-level kernel families using GPT-5.6 Sol, including general matrix multiplications (GEMM) [4], flash attention [19], and mixture-of-experts (MoE) feed-forward networks [62]. On AMD MI300X, these kernels account for up to 90% of GPU execution time during LLM inference [27]. Kernels optimized by Ave achieve

89–99% of the effective throughput of state-of-the-art hand-optimized libraries [1, 4]. Across the evaluated workloads, Ave improves geometric-mean throughput by 1.62–1176× over uncontaminated agentic baselines. For flash attention, Ave outperforms KernelFalcon [70] by 3×. Beyond these kernel families, Ave can also generalize to KernelBench [56]: with in-context DSL examples, it solves all 100 Level 1 and 88 Level 2 tasks within three attempts.

Concretely, we make the following contributions:

- We introduce data-flow invariants for agentic GPU kernel optimization, improving the reliability and performance of generated kernels.
- We design a tile-based DSL with tag functions and assertions, together with a static analysis that validates invariants at compile time with zero runtime overhead.
- We build an agentic optimization harness with an ICRL planner that proposes optimization candidates, a selector that constructs plans, and a lowering agent that implements them and instantiates their invariants in the Ave DSL.
- We evaluate Ave on GEMM, flash attention, MoE, and KernelBench, demonstrating substantial gains over existing agentic baselines.

2 Motivating Example

We use flash attention [19] on the AMD MI300X GPU as a running example to illustrate why high-performance kernel development demands tightly coupled algorithmic and hardware-level reasoning. Together with GEMM and MoE layers, attention accounts for up to 90% of GPU execution time in LLM serving [27], making its optimization critical.

The standard attention mechanism operates on three input matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, where N is the sequence length and d is the head dimension. It computes $\mathbf{S} = \mathbf{Q}\mathbf{K}^T$, applies softmax row-wise to obtain $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, and produces $\mathbf{O} = \mathbf{P}\mathbf{V}$. A naïve implementation materializes the $N \times N$ score matrix $\mathbf{S}$, requiring $\mathcal{O}(N^2)$ memory, prohibitive for long contexts (e.g., N = 256K). Flash attention avoids this by tiling: it loads a $B_r \times d$ block of $\mathbf{Q}$ into registers, then streams through $\mathbf{K}$ and $\mathbf{V}$ in $B_c \times d$ blocks staged in shared memory, accumulating into $\mathbf{O}$ via online softmax [48] to achieve $\mathcal{O}(N)$ memory. The tile sizes B_r and B_c are chosen so the working set fits in registers and shared memory.

GPU Architecture. Ave targets the AMD MI300X GPU, which implements the CDNA3 architecture [2]. Like other modern GPUs, the MI300X follows a Single Instruction, Multiple Threads (SIMT) design. Each Compute Unit (CU) contains in-order execution cores, a shared register file, and Local Data Share (LDS), or shared memory, and is analogous to an NVIDIA streaming multiprocessor. A 64-thread *wavefront* (AMD's warp) executes in lockstep on a 16-lane vector unit. Several wavefronts form a *workgroup* sharing LDS. MI300X also provides MFMA matrix cores, 2-wide vector fused multiply-add instructions, and Vector Accumulation Registers (AGPRs) that expand the available register pool. These features require explicit compiler support.

Achieving peak performance on a GPU requires exploiting this architecture along three axes. First, kernels balance per-workgroup register and shared-memory use against occupancy. Second, MI300X's arithmetic-to-bandwidth ratio of ~250 demands vectorized accesses, shared-memory reuse, and software pipelining to overlap computation with transfers. Third, instruction scheduling and register allocation matter at the assembly level: poor schedules stall in-order pipelines, while excessive register use causes spills [58].

Optimizing Flash Attention on MI300X. Turning flash attention into a kernel reaching peak throughput requires a cascade of hardware-specific optimizations. Figure 1 shows a simplified Ave DSL implementation with 512 threads, d=128, B_r=256, and B_c=64, embodying these optimizations. The pseudocode focuses on layouts and operand pairing, eliding the full pipeline schedule and wavefront-group control flow.

The kernel executes two GEMMs $\mathbf{S} = \mathbf{Q}\mathbf{K}^T$ and $\mathbf{O} = \mathbf{P}\mathbf{V}$ on matrix cores via MFMA instructions. Each 32×32×8 MFMA expects column-major input operands $\mathbf{A}$ and $\mathbf{B}$ and accumulator $\mathbf{C}$ in hardware-specific swizzled layouts. Each thread contributes 4 elements from each input. Adapting row-major attention to these layouts requires operand reordering and transposition. Reaching peak throughput also requires shared-memory reuse, software pipelining to overlap global memory loads with MFMA computation, partitioning data-flow across disjoint wavefront groups, and careful instruction scheduling to keep the in-order cores busy without pipeline stalls while staying within the register budget to avoid spills. For example, AITER's handwritten assembly kernel uses 11 pipeline stages across two wavefront groups. Section 4 details how the DSL expresses these optimizations.

The data shuffling and layout transformations are *error-prone*: mismatches between expected and actual data layouts cause correctness bugs. Ensuring correctness of these global properties is time-consuming when drafting and debugging optimizations. Ave introduces tag functions and data-flow invariants, expressed as `assert` statements in Figure 1, to check that values are correctly paired at each use site. Section 4 describes these mechanisms in detail.

3 Overview of Ave

Ave combines three components to generate and optimize GPU kernels: (i) a tile-based, Pythonic DSL with tag functions and assertions (§4), (ii) compile-time invariant analysis with zero runtime cost (§5), and (iii) an agentic harness guided by invariant feedback (§6). The implementation primarily targets AMD MI300X and generalizes to AMD MI350X and NVIDIA H200 (§9.3).

Ave DSL. The Pythonic, tile-based DSL resembles CuTe and Triton and adds tag functions and assertions for data-flow

```
def attn(d:const,gqa:const,Q:Tensor((sq,h_q,d),bf16),K:Tensor((sq,h_kv,d),bf16),
  V:Tensor((sq,h_kv,d),bf16),O:Tensor((sq,h_q,d),bf16), ...):
  # Assign tags to QKV
  T_Q,T_K,T_V = Q[sq,h_q,d]→(h_q/gqa,d), K[sq,h_kv,d]→(h_kv,d), V[sq,h_kv,d]→(h_kv,sq)
  # Vectorize and tile memory access for MFMA
  idx_q,idx_h,tid = blockIdx.x, blockIdx.y, threadIdx.x
  wid,wtid,nw,r0 = tid/64,tid%64,(wid&2)*2+wid/4*2+wid%2,make_local((4,),u256)
  gQ,rQ = Q.view((sq/Br,Br,h_q,d/8),u128), make_local((8,),u128)
  gK,sK = K.view((sq/Bc,Bc,h_kv,d/8),u128), make_shared((512,2),u128)
  gV,sV = V.view((sq/Bc,Bc,h_kv,d/2),u32), make_shared((2,2,4,4,32),u64)
  s̃K,s̃V = sK.view((2,32,8,2),u128), sV.view((2,2,4,4,16),u128)
  rS,r̃S,tU = make_local((2,),u256),rS.view((4,8),bf16),make_local((2,4,),u32)
  ∀i ∈ [0,8): rQ[i] = gQ[idx_q,wid*32+wtid%32,idx_h,i*2+wtid/32]
  for i in range(sq/Bc):
    for j in range(2):
      sK[tid,j] = gK[i,j*32+tid/16,idx_h/gqa,tid%16]
      # Vectorize V^T via bit rearrangements
      ∀k ∈ [0,4): tU[j,k] = gV[i,j*32+wid*4+k,idx_h/gqa,wtid]
      sV[0,j,wtid/16,nw%4,(wtid%16)*2+nw/4] = concat^3_{k=0}(tU[j,k]_lo)
      sV[1,j,wtid/16,nw%4,(wtid%16)*2+nw/4] = concat^3_{k=0}(tU[j,k]_hi)
    syncthreads()
    T_r̃S = r̃S[x,y]→(idx_h/gqa,i*Bc+x/2*32+x%2*16+y/4*8+wtid/32*4+y%4)
    for j in range(16):
      tK,tQ = s̃K[j/8,wtid%32,j%8,wtid/32], rQ[j%8]
      t̃K,t̃Q = tK.view((8,),bf16), tQ.view((8,),bf16)
      assert tag(t̃K[...]) == tag(t̃Q[...]) # t̃Q and t̃K should match
      rS[j/8] = matmul(t̃K,t̃Q,...) # FP32 accumulate and pack S=QK^T
    ... # Online softmax: overwrite rS with packed P and rescale rO
    for j in range(16): # PV, V is transposed due to MFMA
      tV = s̃V[wtid%2,j/8,j%4,j%8/4*2+wtid/32,(wtid/2)%16]
      t̃V = tV.view((8,),bf16)
      assert tag(t̃V[...]) == tag(r̃S[...]) # t̃V and r̃S should match
      rO[j%4] = matmul(t̃V,r̃S[j/4],...) # FP32 accumulate and pack O
    syncthreads()
  ... # Normalize, pack, and store O
```

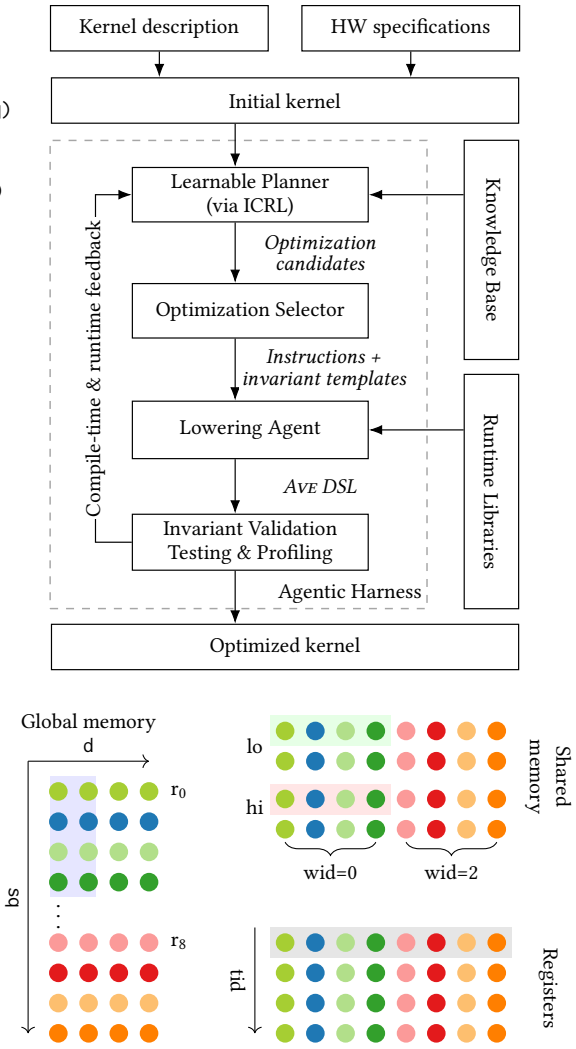


Figure 1. Overview of AVE. Left: Simplified DSL implementation of flash attention (d=128, B_r=256, B_c=64, 512 threads). Top right: AVE's workflow for agentic optimization. Bottom right: Tag propagation for V across memory levels. Each color represents a unique sequence-coordinate tag, and background shading links code regions to memory access patterns. r_0 and r_8 represent row 0 and row 8.

invariants. It exposes hardware instructions and compiler policies while abstracting complex memory layout encodings as tiles (§4).

Invariant Validation. AVE validates data-flow invariants at compile time with zero runtime overhead. The compiler lowers the DSL to MLIR [40] and propagates tags across assignments and shared memory using flow-sensitive, path-insensitive analysis. Path-insensitive merging keeps solving tractable. An SMT solver checks tag assertions and returns concrete assignments that witness violations and guide the agent [20] (§5).

Agentic Optimization Harness. AVE adopts an agentic workflow to iteratively optimize kernels, as shown at the top right of Figure 1. Starting from a DSL kernel and unit tests, a planner retrieves optimization knowledge, a selector orders compatible proposals, and a lowering agent implements them. Validation checks invariants and tests before profiling. Invariant violations and performance update the benchmark-local planner through ICRL (§6).

4 The AVE DSL

Layouts and Tiles. The AVE DSL models all memory accesses via *tiles* [24, 41, 52, 69]. A tile is a memory region paired with a layout function [10] that maps multi-dimensional logical coordinates to one-dimensional physical offsets. A layout function $\mathcal{L}_{(\mathbf{s},\mathbf{t})}$ is parameterized by two tuples: shapes $\mathbf{s} = (s_1, \ldots, s_n)$ and strides $\mathbf{t} = (t_1, \ldots, t_n)$. It maps an n-element coordinate $\mathbf{c} = (c_1, \ldots, c_n)$ to a linear index. Following the

CuTe layout algebra, when all elements of **s** and **t** are integers, the layout computes

$$\mathcal{L}(c_1, \ldots, c_n) = \sum_{i=1}^{n} c_i t_i$$

For a contiguous row-major tensor, $t_i = \prod_{j=i+1}^{n} s_j$: each stride is the product of the dimensions that follow it, and $t_n = 1$. For brevity, these strides are omitted. For example, the `Q:Tensor((sq,h_q,d),bf16)` in Figure 1 has the layout

$$\mathcal{L}^{\mathtt{Q}}(c_1, c_2, c_3) = c_1 \mathtt{h_q d} + c_2 \mathtt{d} + c_3.$$

The subscript operator in Figure 1 maps coordinates to a linear index and accesses the corresponding memory location.

Following CuTe, a nested layout represents its shape, stride, and natural coordinate as congruent hierarchical tuples: every coordinate leaf pairs with the stride leaf at the same position. A scalar coordinate for a nested mode is first converted to this hierarchical form by mixed-radix division and remainder using the mode's shape. The layout recursively sums the coordinate–stride products. For example, for the first nested coordinate $\mathbf{c}_1 = (c_{1,1}, c_{1,2})$, with shape $(s_{1,1}, s_{1,2})$ and stride $(t_{1,1}, t_{1,2})$, the corresponding sub-layout computes $\mathcal{L}_1(\mathbf{c}_1) = c_{1,1}t_{1,1} + c_{1,2}t_{1,2}$. Nested layouts primarily represent hardware-swizzled layouts used by tensor cores and matrix cores. Layout algebras further define composition, logical division, and logical projection over layout functions to represent operations such as data shuffling and dispatching disjoint work to warps (wavefronts on AMD) and threads. We refer the reader to [10] for details.

DSL programs use `make_local()` and `make_shared()` to allocate a tile backed by registers and shared memory. DSL programs may use the `view()` function to reinterpret a tile under a different logical layout. For example, line 8 of Figure 1 derives `gQ` to logically partition `Q` by B_r and access data in 16-byte chunks:

```
gQ = Q.view((sq/Br,Br,h_q,d/8),u128)
```

Vectorizing loads in 16-byte chunks is a common optimization to fully utilize GPU memory bandwidth. However, Ave requires tiles to express all memory accesses and restricts layout operations. For example, `view()` requires source and destination layouts to cover identical memory sizes. These constraints simplify program analysis and eliminate the need for full-scale alias analysis.

Continuing with the flash attention example in Figure 1, the kernel maps work across 8 wavefronts. Each wavefront owns 32 rows of **Q** and produces the corresponding rows of **O**, while all wavefronts share $B_c \times d$ tiles of **K** and **V** in shared memory. MFMA expects column-major input operands **A** and **B** and accumulator **C**. In attention, **S** must be row-major because **P**=softmax(**S**) is computed row-wise, and **O** must also be row-major. The kernel therefore swaps **Q** and **K** for $\mathbf{QK}^T$ and transposes **V** during staging for **PV**. The `matmul` helper combines two MFMAs along the reduction dimension to perform a 32×32×16 operation per wavefront. Each thread holds 8 BF16 elements from each input and 16 FP32 accumulator elements.

In Figure 1, `matmul` accumulates **S** in FP32 and packs it into the BF16 `rS` tile. The elided online softmax computes **P** row-wise, overwrites `rS` with the packed probabilities, and rescales `rO`. To transpose **V**, each thread loads a 4×2 subtile into `tU` (line 18) and scatters its low and high halves into transposed positions in `sV` (lines 19–20). This organization uses 4-byte global loads, 8-byte shared-memory stores, and 16-byte swizzled shared-memory loads that feed the **PV** loop directly (lines 29–33, highlighted in gray). `matmul` accumulates the output in FP32 and packs it into `rO`. After all sequence tiles, the kernel normalizes, packs, and stores **O**. The final steps and lane-to-output mapping are elided in the figure.

Tag Functions and Assertions. A tag function annotates tensor elements with symbolic tags: tuples of constants, symbolic expressions of coordinates, or the special symbols $\bot$ and $\top$. Tags propagate through assignments.

For example, $\mathtt{T_Q}$, $\mathtt{T_K}$, and $\mathtt{T_V}$ in Figure 1 assign tags to elements of `Q`, `K`, and `V` based on their coordinates (the type `bf16` is omitted for brevity):

$$\mathtt{T_Q} = \mathtt{Q[sq,h_q,d]} \to \mathtt{(h_q/gqa,\ d)}$$
$$\mathtt{T_K} = \mathtt{K[sq,h_{kv},d]} \to \mathtt{(h_{kv},\ d)}$$
$$\mathtt{T_V} = \mathtt{V[sq,h_{kv},d]} \to \mathtt{(h_{kv},\ sq)}$$

Tag assertions require conformity or non-conformity between tags. The assertion on line 26 in Figure 1 requires matching tags for elements of `Q` and `K`:

$$\mathtt{assert\ tag}(\widetilde{\mathtt{tK}}[\ldots]) == \mathtt{tag}(\widetilde{\mathtt{tQ}}[\ldots])$$

Computing $\mathbf{S} = \mathbf{QK}^T$ requires each wavefront to multiply its 32 rows of **Q** with two 32×128 blocks of **K** per iteration. The tag functions $\mathtt{T_Q}$ and $\mathtt{T_K}$ return the attention-head and reduction-dimension indices, so their tags match only when the operands belong to the same head and represent the same position along the reduction dimension. A passing assertion confirms that the implementation preserves this operand-pairing requirement across optimizations such as shared-memory staging or software pipelining. Similarly, the tag functions $\mathtt{T_V}$ and $\mathtt{T}_{\widetilde{\mathtt{rS}}}$ return the attention-head and reduction-dimension indices. The assertion on line 32 therefore checks that the operands of $\mathbf{O} = \mathbf{PV}$ belong to the same head and represent the same position along their shared reduction dimension, without equating their independent output dimensions. Figure 1 (bottom right) visualizes `V`'s sequence-coordinate tags as data propagate from global memory to shared memory and registers.

Compiler and Hardware Primitives. The DSL exposes compiler and hardware primitives that enable agents to explore low-level optimizations. The DSL operates at the thread level: kernels manage thread indices and synchronization (e.g.,

`syncthreads()`) directly, enabling the topology-aware dispatch optimizations [69].

Ave also exposes the eager materializations and compiler scheduling barriers for tuning instruction scheduling in the backend. These compiler barriers constrain instruction placement around runtime synchronization primitives such as `s_barrier`. Unlike `s_barrier`, they emit no synchronization instruction. These interfaces provide LLMs explicit control over hardware features that are critical for reaching peak performance on GPUs [33, 67].

Additionally, Ave exposes vectorized instructions for explicit use in the DSL. For example, the AMD MI300X issues one 2-way vectorized FMA per cycle on packed registers, doubling scalar FMA throughput. The AMD GPU backend can emit these instructions, but suboptimal register allocation often prevents it, so explicit vectorization in the DSL sidesteps this limitation. Ave further exposes instructions such as `buffer_load_dwordx4` for branchless memory access with hardware out-of-bounds (OOB) protection, and allows selecting the accumulator register class (AGPR) directly in buffer load and MFMA instructions, practically doubling available vector registers over off-the-shelf toolchains.

5 Validating Data-Flow Invariants

Ave validates data-flow invariants by tracking tags through control and data flow at compile time. The compiler lowers DSL programs to MLIR and performs a flow-sensitive, path-insensitive analysis that propagates tags through assignments and shared-memory accesses. At control-flow joins, Ave merges incoming tags in a lattice with the partial order $\bot < t < \top$ for every concrete tag t:

$$\text{merge}(t_1, t_2) = \begin{cases} t_1 & \text{if } t_2 \le t_1 \\ t_2 & \text{if } t_1 < t_2 \\ \top & \text{otherwise} \end{cases}$$

Constants carry $\bot$. Consider the guarded selection `rQ = 0 if isOOB(...) else Q[...]`, where `isOOB` prevents an out-of-bounds access. The zero-valued branch contributes $\bot$, so `rQ` retains the tag of `Q[...]` because $\text{merge}(\bot, t) = t$. Conversely, merging different concrete tags yields $\top$, denoting non-conformity. Accordingly, `==` requires the merge to be below $\top$, whereas `!=` requires $\top$: $\bot$ conforms with any concrete tag, while $\top$ does not conform even with itself. Ave can reset shared-memory tags to $\bot$, permitting storage to be reused safely across pipeline stages without disabling subsequent tag tracking.

The analysis tracks memory at byte granularity. This granularity covers the evaluated byte-aligned accesses and supports packed storage for formats such as `mxfp4` and `nvfp4`, although we do not evaluate FP4 kernels. Ave models global and shared memory as typed chunks and automatically propagates tags across accesses of different widths, such as loads vectorized into 16-byte chunks. To keep the analysis bounded, it does not model the full heap or track tags written to global memory. The trade-off is that Ave cannot express invariants depending on heap-resident values. For example, in MoE sorting, where global `<expert,token>` tuples are grouped by expert ID, Ave cannot assert that each tuple reaches the correct bucket in shared memory. Once data is in shared memory or registers, Ave can assign address-derived labels and track transformations such as swizzling. Multi-GPU MoE kernels such as COMET [77] exchange data via global cross-GPU buffers, which Ave does not currently track. Modeling these buffers as explicit producer–consumer queues remains future work.

Ave encodes propagated tags and tag assertions as satisfiability modulo theories (SMT) constraints and uses the solver to reason about the resulting layout algebra. The goal is to validate data-flow invariants, not to construct full formal proofs. In particular, path-insensitive merging avoids enumerating branch conditions and keeps analysis tractable for optimized kernels. When an assertion fails, the solver returns a concrete assignment witnessing the violating abstract state, including values for symbolic program variables such as thread IDs, loop indices, and tensor coordinates.

Tags are purely *compile-time constructs*: all tracking and validation occur within the compiler and incur no runtime overhead. The current prototype requires workgroup dimensions known at compile time, allowing Ave to model registers as a fixed-size array indexed by thread ID. It also unrolls loops before analysis.

6 Agentic Optimization Harness

The Ave harness starts from a compile-and-test-passing DSL kernel and retains the fastest candidate that passes automated invariant checks and unit tests. In each round, a planner proposes optimization candidates, a selector constructs an ordered plan, and a lowering agent implements it. Only a measured speedup replaces the current kernel.

Persistent Expert Knowledge. The knowledge base $\mathcal{K}$ contains auditable, expert-curated hardware facts, DSL references, and optimization recipes. Recipes specify applicability, transformations, hardware controls, and invariant templates. We curate entries from KernelWiki-style documentation [36], vendor examples, public pull requests, and microbenchmarks. Retrieve filters them by the request, GPU, and current kernel to produce K_t.

Local Planner State. For a request denoted by q, the fixed LLM π_ϕ uses a benchmark-local prompt $\theta_{q,t}$ updated after each optimization round through in-context reinforcement learning (ICRL) [26, 49]. In round t, it observes $x_t = \langle q, s^\star, K_t, F \rangle$: the best kernel $s^\star$, retrieved knowledge K_t, and feedback history F. The initial history entry contains the initial kernel s_0 and its profile. Later entries record candidates, validation feedback, and profiles. The LLM produces ranked proposals P_t specifying optimizations, preconditions, target regions,

invariant templates, and expected benefits. SELECT orders compatible proposals into $a_t = (a_{t,1}, \ldots, a_{t,m_t})$.

Lowering and Validation. LOWER invokes the lowering agent to apply every optimization in a_t to $s^\star$. The agent binds each selected instruction to code, specializes its parameters, and materializes its invariant templates as tag functions and assertions, returning $\hat{s}_t$. Validation compiles this candidate, checks every generated invariant (the set may be empty), and runs unit tests. VALIDATE returns compilation status C_t, source-level assertion sites A_t, failed sites V_t, test status U_t, and record f_t. For a passing candidate, PROFILE returns its latency and appends the runtime profile to f_t. Appending f_t to F gives the next round its transformation and outcome.

Reward Function. Let $C_t \in \{0, 1\}$ indicate successful compilation and $U_t \in \{0, 1\}$ indicate that all unit tests pass. Let A_t be the source-level assertion sites in $\hat{s}_t$, and let $V_t \subseteq A_t$ be those with at least one failing check. Thus, compiler unrolling does not change the number of assertions. If compilation succeeds and $A_t \neq \emptyset$, the invariant score is $I_t = 1 - |V_t|/|A_t|$. Otherwise, $I_t = 0$. If compilation fails, invariant checking does not run, and we define $V_t = \emptyset$. Unit tests run only when $C_t = 1$ and $V_t = \emptyset$. If they do not run, $U_t = 0$. For a kernel that compiles, passes every invariant check, and passes all unit tests, let $\hat{\ell}_t$ be its measured latency and $\ell^\star$ the latency of the current best kernel. Runtime improvement is defined as:

$$\Delta_t = \begin{cases} \dfrac{\ell^\star - \hat{\ell}_t}{\ell^\star}, & C_t = 1 \ \wedge \ V_t = \emptyset \ \wedge \ U_t = 1, \\ 0, & \text{otherwise.} \end{cases}$$

Thus, a positive Δ_t denotes lower latency, while a validated slowdown receives a negative performance term. The complete reward is $R_t = C_t + I_t + U_t + \Delta_t$. The replay record stores the planner input x_t, candidate record f_t, validation outcomes C_t, A_t, V_t, and U_t, performance term Δ_t, and reward R_t. We denote this record by d_t.

Text-Gradient Planner Update. Algorithm 1 summarizes the planner update. After each optimization round, POLICYEVAL summarizes d_t as E_t, ANALYZE converts it into a textual critique g_t, and PARAMETERUPDATE applies a constrained text-gradient edit to produce $\theta_{q,t+1}$ [74]. Neither the LLM weights nor $\mathcal{K}$ change.

Example. We provide a concrete example to illustrate the process. On AMD MI300X, hardware entries cover MFMA, buffer operations, data parallel primitive (DPP) instructions, and LDS, vector general-purpose register (VGPR), and AGPR limits. The omitted `matmul` helper in Figure 1 uses two 32×32×8 BF16 MFMA instructions for a 32×32×16 operation. For row-major QK^T, it swaps Q and K as detailed in §4. Figure 1 keeps Q in `rQ` and stages K in `sK`, read through $\widetilde{\texttt{sK}}$. The planner targets this region with a recipe whose templates define the Q/K tags and operand assertion. Here, a_t contains only this

Algorithm 1: ICRL-guided optimization.

Input: Query q, initial compile-and-test-passing kernel s_0, fixed knowledge base $\mathcal{K}$, fixed LLM π_ϕ, initial prompt $\theta_{q,0}$, optimization rounds N

$s^\star \leftarrow s_0$; $(\ell^\star, f_{-1}) \leftarrow \textsc{Profile}(s_0, \langle s_0 \rangle)$; $F \leftarrow \langle f_{-1} \rangle$;
for $t = 0, 1, \ldots, N-1$ **do**
  $K_t \leftarrow \textsc{Retrieve}(\mathcal{K}, q, s^\star)$;
  $x_t \leftarrow \langle q, s^\star, K_t, F \rangle$;
  $P_t \leftarrow \pi_\phi(x_t; \theta_{q,t})$;
  $a_t \leftarrow \textsc{Select}(P_t)$;
  $\hat{s}_t \leftarrow \textsc{Lower}(s^\star, a_t)$;
  $f_t \leftarrow \langle a_t, \hat{s}_t \rangle$;
  $(C_t, A_t, V_t, U_t, f_t) \leftarrow \textsc{Validate}(\hat{s}_t, f_t)$;
  $(I_t, \Delta_t) \leftarrow (0, 0)$;
  if $C_t = 1 \wedge A_t \neq \emptyset$ **then**
    $I_t \leftarrow 1 - |V_t|/|A_t|$;
  end
  if $C_t = 1 \wedge V_t = \emptyset \wedge U_t = 1$ **then**
    $(\hat{\ell}_t, f_t) \leftarrow \textsc{Profile}(\hat{s}_t, f_t)$;
    $\Delta_t \leftarrow (\ell^\star - \hat{\ell}_t)/\ell^\star$;
  end
  $(R_t, F) \leftarrow (C_t + I_t + U_t + \Delta_t, F \parallel \langle f_t \rangle)$;
  $d_t \leftarrow \langle x_t, f_t, C_t, A_t, V_t, U_t, \Delta_t, R_t \rangle$;
  if $\Delta_t > 0$ **then**
    $(s^\star, \ell^\star) \leftarrow (\hat{s}_t, \hat{\ell}_t)$;
  end
  $E_t \leftarrow \textsc{PolicyEval}(d_t)$;
  $g_t \leftarrow \textsc{Analyze}(E_t)$;
  $\theta_{q,t+1} \leftarrow \textsc{ParameterUpdate}(\theta_{q,t}, g_t)$;
end

Output: Best kernel $s^\star$

action. Lowering preserves existing assertions and adds this one with the prompt:

> Tile the scalar QK^T dot products into 32×32×16 accumulations, using two 32×32×8 BF16 MFMA instructions per accumulation. Pass BF16 views of `tK` and `tQ` to `matmul`, which accumulates in FP32 and packs the result into BF16 scores.

The lowering agent materialized the tags and assertion and staged K in `sK`, but formed `tK` from the original $(512, 2)$ layout. The kernel compiled, but the tags of $\widetilde{\texttt{tK}}$ and $\widetilde{\texttt{tQ}}$ disagreed at the matmul use site, so tests and profiling did not run. Thus, $(C_t, U_t, \Delta_t) = (1, 0, 0)$, $I_t < 1$, and $R_t = 1 + I_t < 2$. The validator added the following witness to f_t, which the harness subsequently appended to F:

> Assertion failed at `attn:26`: tag($\widetilde{\texttt{tK}}$`[0]`) ≠ tag($\widetilde{\texttt{tQ}}$`[0]`).
> Witness: i=0, j=1, tid=0, wid=wtid=0, idx_q=idx_h=0.

The witness identified the mismatched MFMA operands. POLICYEVAL summarized d_t, including the candidate code and witness, as E_t. ANALYZE then traced the mismatch to the flattened LDS access and produced the critique g_t. PARAMETERUPDATE applied g_t to produce $\theta_{q,t+1}$. In the next round, the planner proposed the following revision, which SELECT placed at $a_{t+1,1}$:

Stage K in sK with 16-byte cooperative loads, synchronize, and reinterpret its $(512, 2)$ layout as $(2, 32, 8, 2)$ u128. For MFMA-loop index $j \in [0, 16)$, let $l =$ wtid. Read tK from $\widetilde{\text{sK}}$ at $(j/8, l\%32, j\%8, l/32)$, pair it with tQ = rQ[$j\%8$], and view both as 8 BF16 elements. Update rS[j/8] with matmul, which accumulates these views in FP32 and packs the result into rS[j/8].

The revised binding reads K through $\widetilde{\text{sK}}$ and leaves rQ unchanged. It passed all checks and was profiled.

7 Implementation

The Ave prototype comprises approximately 22,000 lines of C++ and 7,000 lines of Python across the DSL compiler, program analysis, agentic harness, and JIT infrastructure. We use Codex as the harness runtime [54] and configure the planner, lowering agent, PolicyEval, Analyze, and ParameterUpdate with the same selected model. Ave retains the current best kernel and summaries of prior candidates, validation outcomes, and profiles. The harness periodically compacts the feedback history. The Python front-end parses DSL programs into an AST, which the C++ compiler lowers to an extended MLIR dialect encoding layout algebras, then to LLVM IR and GPU executables. We extend the LLVM 20 AMDGPU backend with explicit synchronization barrier scheduling, register-class selection for buffer load and MFMA instructions, and eager materialization through phantom uses in inline assembly.

Tag propagation and invariant checking use custom passes built on MLIR's pass infrastructure. These passes traverse the IR and emit SMT constraints encoding data-flow invariants, which Z3 solves [20].

The JIT layer follows Triton's model: generics propagate compile-time constants into GPU kernels, compiled binaries are cached, and Python bindings expose kernel invocation.

8 Discussion

Limitations of Data-Flow Invariants. Data-flow invariants validate concrete transformations but do not propose algorithmic changes, such as restructuring standard attention into tiled flash attention with online softmax [19], or selecting search-space parameters such as tile sizes and warp/wavefront specialization. The planner and knowledge base handle these decisions. Applying invariants while the algorithm is still changing could also constrain exploration. A full optimization can be separated into two stages: algorithm selection and initial kernel generation occur first, after which data-flow invariants guard low-level optimizations.

Full formal verification can establish stronger properties, but requires substantial proof engineering [7, 31, 37] and produces implementation-specific proofs. Ave instead checks shared-memory layouts and operand pairing with tractable, flow-sensitive, path-insensitive analysis. As Section 5 details, properties involving global-memory writes, such as MoE routing and multi-GPU communication, require new abstractions. The tensor memory (TMEM) extension discussed below illustrates one such extension [53]. Additionally, the analysis tracks memory at byte granularity. It treats two packed FP4 values within a byte as one unit, so permutations of individual nibbles are invisible to data-flow invariants.

Using More Tokens and Stronger Models. As shown in Section 9, stronger models often improve generated GPU kernels, but higher token use alone does not close the gap with expert libraries: without invariants, lowering agents can still exhaust their contexts through trial-and-error repairs.

Unguided agentic optimization is vulnerable to reward hacking, which can inflate measured speedups without yielding useful kernels [30]. Our manual inspection is qualitative and targets known shortcut patterns that bypass the main computation, including PyTorch fallbacks, precomputed or zeroed outputs, and branches on benchmark values. We do not classify semantics-preserving changes within a convolution kernel, such as strength reduction for fixed filter dimensions, as reward hacking. Data-flow invariants additionally reject shortcuts that violate declared use-site relationships. Neither mechanism is exhaustive: inspection may miss subtler shortcuts outside known patterns, and invariants cannot detect behavior outside their vocabulary.

Compiler and Language Design. Higher-level DSLs such as Triton automate layouts and thread scheduling, eliminating several manual optimizations. Peak performance still requires low-level controls such as inline assembly, explicit register-class selection, and scheduling barriers [1, 67]. Because compiler stacks often lag behind new hardware, vendors rely on leaky language abstractions [67], or just ship hand-optimized assembly. Ave exposes these controls as policies chosen by LLMs and implemented by the compiler.

Ave remains syntactically close to existing DSLs so that LLMs can generalize from their training data via in-context learning [49], enabling adoption of new frontier models without retraining. Better compiler automation could absorb more low-level decisions, while a more radical LLM-oriented language could improve token efficiency and support stronger code generation.

Generalizing across Hardware Architectures. Porting Ave to a new GPU requires (i) integrating the target toolchain and exposing its low-level intrinsics and (ii) adding hardware-specific optimizations to the knowledge base. ICRL binds these entries to new kernels without model retraining, but experts must encode new techniques.

We added NVIDIA Hopper and AMD CDNA4 backends with localized changes. Initial support for both architectures took approximately one person-week. For CDNA4, we exposed FP4 intrinsics. For Hopper, we exposed warp-group matrix multiply-accumulate (WGMMA) and mbarrier. We then added architecture-specific knowledge entries. NVIDIA Blackwell requires one further abstraction because its kernels

Method	GPT-5.6 Sol					DeepSeek V4 Pro					DeepSeek V4 Flash				
	Perf	Time	Turns	Tok.	Cost	Perf	Time	Turns	Tok.	Cost	Perf	Time	Turns	Tok.	Cost
GEMM															
Ave	0.89	2552/185/107	12	0.70	$6.49	0.88	2809/191/96	17	0.93	$1.16	0.90	3947/187/112	13	1.00	$0.61
KernelFalcon	0.55	1507/81/337	12	0.14	$2.92	0.39	1327/137/353	12	0.10	$0.23	0.49	306/21/51	8	0.05	$0.04
KSearch	0.36	802/127/209	12	0.16	$2.09	0.06	3399/220/97	20	0.41	$0.81	0.30	5621/326/117	20	0.86	$0.71
CUDAForge	0.07	752/294/919	11	0.11	$1.86	0.01	1789/159/512	19	0.17	$0.34	0.03	489/238/776	17	0.13	$0.08
Atrex	0.51	11201/1/465	327	60.56	$46.26	0.24	9980/0/123	524	105.54	$8.17	0.88	2033/13/295	207	22.64	$0.56
Attention															
Ave	0.99	1999/16/63	6	0.44	$4.67	0.94	1175/14/66	5	0.35	$0.51	0.94	2569/18/67	7	0.70	$0.40
KernelFalcon	0.33	1570/215/336	12	0.14	$2.93	0.09	1958/50/175	13	0.13	$0.27	0.10	523/23/54	10	0.11	$0.07
KSearch	0.15	1237/121/130	19	0.31	$4.06	0.03	2891/211/130	14	0.34	$0.68	0.03	3812/31/133	10	0.62	$0.51
CUDAForge	0.001	681/271/896	11	0.12	$2.08	0.01	1648/344/1123	14	0.20	$0.44	0.01	615/187/314	17	0.17	$0.10
Atrex	0.91	9477/2/660	347	49.68	$34.55	0.68	8524/6/153	356	60.47	$4.35	1.01	4350/6404/1254	720	112.21	$1.57
Fused MoE															
Ave	0.94	2651/17/57	9	0.75	$7.01	0.91	2400/26/80	10	0.77	$1.10	0.92	3416/67/224	11	1.02	$0.56
KernelFalcon	0.12	725/23/54	8	0.11	$1.80	0.0002	1282/130/321	12	0.18	$0.29	0.02	764/123/231	13	0.16	$0.10
KSearch	0.07	2111/113/110	14	0.39	$5.71	Fail	5829/484/579	20	0.83	$1.38	0.002	937/379/367	14	0.27	$0.14
CUDAForge	0.02	676/272/1375	10	0.11	$2.03	0.001	1135/156/357	19	0.16	$0.30	Fail	4320/461/640	20	0.70	$0.57
Atrex	0.32	14084/14/670	265	196.98	$146.77	Fail	9723/61/2129	1170	145.34	$9.20	0.01	7706/6/4	556	193.98	$2.71

Table 1. Optimization performance and resource costs across workloads, frameworks, and models.

dynamically manage TMEM. We plan to model TMEM as shared memory using a simple lifetime heuristic: optimized kernels typically allocate TMEM near entry and release it near exit. This abstraction would propagate tags through TMEM reads and writes without modifying the core analysis.

9 Evaluation

Our evaluation answers the following research questions:

- *Q1.* How effectively does Ave optimize high-performance GPU kernels? (§9.1)
- *Q2.* How do ICRL and data-flow invariants affect an agent's ability to apply optimizations? (§9.2)
- *Q3.* How well does Ave generalize across kernel families and GPU architectures? (§9.3)
- *Q4.* How do data-flow invariants affect optimization success and cost? (§9.4)

Experimental Environment. Our primary server has two 64-core AMD EPYC 9554 CPUs, 2 TB of DDR5, and eight 192 GB AMD MI300X GPUs. It runs Ubuntu 22.04.5, Linux 5.15.0, and ROCm 7.2.2 [5], whose LLVM 20.0 fork underlies our compiler extensions. Our portability server has two 48-core Intel Xeon Platinum 8558 CPUs, 2 TB of DDR5, and eight 141 GB NVIDIA H200 GPUs. It runs Ubuntu 24.04.1, CUDA 12.8, and driver 570.148.08. We warm up each kernel for one second and report its mean latency over at least 100 runs within GPU graphs, pinning each process to the GPU-local CPU socket. We evaluate all agentic frameworks with GPT-5.6 Sol (GPT) [55], DeepSeek V4 Pro (DSv4) [23], and DeepSeek V4 Flash (DSv4F) [23], adapting their prompts for MI300X. For Ave, the selected model drives every Codex-based harness stage, including planning, lowering, policy evaluation, analysis, and prompt updates.

Turns, Tok., Cost, and Time are averages over attempted problems. Pass@k generation entries are per attempt. A *turn* is an LLM-generation segment between tool calls. Tok. is in millions, Cost is the API charge in U.S. dollars, and Time splits LLM/compile/evaluate wall time in seconds. DeepSeek Cost averages on- and off-peak prices.

Unless noted, throughput is normalized to hipBLASLt for MI300X GEMM, AITER for MI300X attention and MoE, and the PyTorch reference supplied by KernelBench. We consider a kernel valid if it compiles, passes all invariant checks, correctness tests, and a manual reward-hacking review (§8). Only valid kernels contribute to performance. Perf. and Mean are geometric means of normalized throughput. Min and Max are extrema, and >1× counts ratios above parity. Pass@k counts tasks solved within k attempts. KernelBench generation is a separate first phase that stops at the first test-passing candidate. Optimization starts from that candidate. Post-hoc rejection counts as failure without resuming. Fail means no valid implementation while we report the costs for references. Success requires every requested transformation.

	Ave	hipBLASLt	HipKittens	AITER	Triton	Atrex	KernelFalcon	KSearch	CUDAForge
Global intrusive changes									
Software pipelining	✓	✓	✓	✓	✓	✓	✓	-	-
Split K	-	-	-	✓	-	-	-	-	-
MFMA matrix multiplication	✓	✓	✓	✓	✓	✓	✓	✓	-
Stagger K	✓	✓	✓	✓	✓	-	-	-	-
Async memcpy	✓	✓	✓	✓	-	✓	-	-	-
Local source changes									
Tile-size specialization	✓	✓	✓	✓	✓	-	✓	-	-
Bank conflict mitigation	✓	✓	✓	✓	✓	✓	-	-	✗
Vectorized loads	✓	✓	✓	✓	✓	✓	✓	-	-
Loop unrolling	✓	✓	✓	✓	✓	✓	✓	✓	-
Workgroup swizzling	✓	✓	✓	✓	✓	-	✓	-	-
ISA-specific optimizations									
HW OOB-guarded loads	✓	✓	✓	✓	-	✓	-	-	-
Use AGPRs	✓	✓	✓	✓	-	✓	-	-	-
Explicit instruction scheduling	✓	✓	✓	✓	-	✓	-	-	-

Table 2. GEMM optimizations implemented by Ave and other agentic frameworks using GPT-5.6 Sol, alongside hand-optimized libraries. ✓, -, and ✗ denote implemented, absent, and incorrectly implemented optimizations.

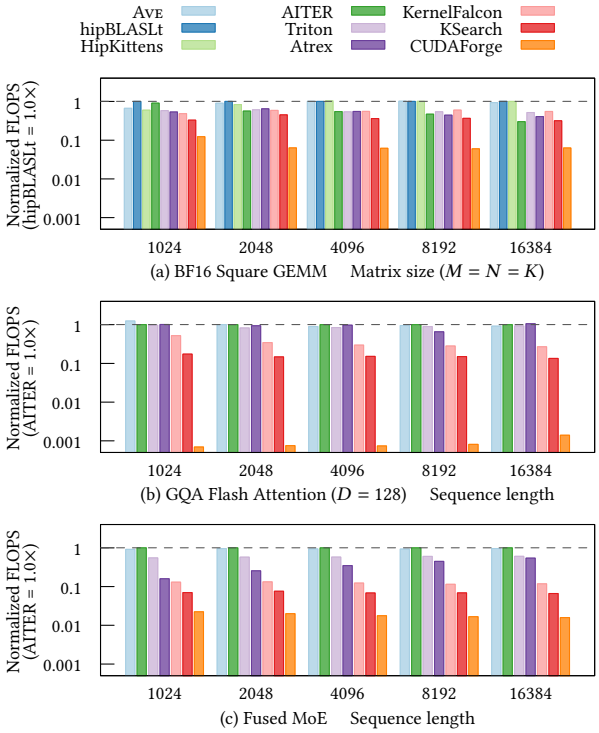


Figure 2. Optimization performance on MI300X using GPT-5.6 Sol. Dashed lines denote parity. Vertical axes are logarithmic.

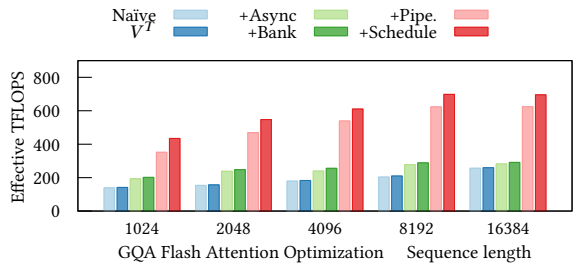
9.1 Kernel Performance

To answer *Q1*, we evaluate five configurations each of three LLM-inference kernels. GEMM multiplies square BF16 matrices with normally distributed entries. Flash attention uses causal BF16 grouped-query attention (GQA) with batch 16, eight query heads, one key-value head, and head dimension 128, matching LLaMA-3 70B prefill under eight-way tensor parallelism. Fused MoE uses DeepSeek-V3.2's block-quantized FP8 configuration [22]: model dimension 7168, intermediate dimension 2048, top-k 4, 32 experts per GPU, and expert parallelism of eight. Each workload starts from a naïve implementation. Table 1 covers the complete optimization process, including costs for failed attempts.

Expert-optimized baselines include hipBLASLt [4], HipKittens [33], AITER [1], and Triton [66]. Agentic baselines include Atrex [72], KernelFalcon [70], KSearch [9], and CUDAForge [78]. We run Ave, KernelFalcon, KSearch, and CUDAForge for up to ten optimization trials and Atrex for up to five because of its API cost, reporting the best valid result. Each trial is one optimization round and may contain multiple LLM turns. These limits may leave some opportunities unexplored. In our experiments, we adopt hipBLASLt from ROCm 7.2.2, AITER 0.1.10post3, and the ROCm Triton fork 3.6.0+rocm7.2.2.git4ed88892. For each workload, the Triton baseline adopts the faster available Triton implementation from HipKittens or AITER. HipKittens GQA lacks MI300X support. We enable available autotuning.

In Figure 2, hipBLASLt for GEMM and AITER for attention and MoE serve as the expert references for each workload. With GPT, Ave's normalized throughput is 0.89–0.99× relative to these references and 1.62–1176× relative to uncontaminated agentic baselines. Atrex uses many more turns than Ave and the other agentic frameworks because it relies on coding agents such as Codex and Claude Code for long optimization trajectories.

GEMM. Table 2 groups optimizations by implementation difficulty for an LLM, not performance impact. Global intrusive changes are hardest because they coordinate control flow, data movement, and layouts across a kernel. These include pipelining, Split K, MFMA layouts, Stagger K for memory-controller hotspots [3], and asynchronous copies. Local changes need only small patches for tile specialization, bank-conflict mitigation, vectorization, unrolling, or workgroup swizzling. ISA-specific optimizations require architectural knowledge and low-level support for hardware OOB protection, AGPRs, and explicit scheduling. AMD intrinsics cover guarded loads and scheduling, while AGPR allocation requires assembly or whole-kernel register management, as in HipKittens.



(a) Optimization performance.

Ablation	Optimization outcome	TFLOPS
(i) Runtime feedback only	+Async and +Bank ineffective	190
(ii) +KB +invariants, -ICRL	$\mathbf{V}^\mathsf{T}$, +Async, +Bank, +Schedule attempted (+Pipe not reached)	255
(iii) +KB +ICRL, -invariants	$\mathbf{V}^\mathsf{T}$ fails	194
(iv) Full Ave	All five applied	588

(b) Harness ablations.

Figure 3. Flash attention optimization. Panel (a) applies stages cumulatively in legend order. Panel (b) reports geometric-mean effective TFLOPs across sequence lengths.

Ave's normalized throughput is 0.89× relative to hipBLASLt. Atrex and KernelFalcon are the strongest agentic competitors, while among the agentic baselines, KernelFalcon is the only framework that completes every workload with every model. Among direct CUDA/HIP frameworks, only KSearch requests and uses MFMA.

KernelFalcon relies on Triton for pipelining, vectorization, unrolling, and tile autotuning, but lacks Stagger K and low-level scheduling. Ave's normalized throughput is 1.62× relative to KernelFalcon and 1.76× relative to Atrex's CuTe-like FlyDSL [41] kernel. Atrex applies most listed transformations but misses workgroup swizzling, while incomplete FlyDSL autotuning prevents tile specialization. Ave's normalized throughput is also 1.6× relative to expert Triton. The evaluated ROCm Triton baseline cannot express asynchronous copies, explicit AGPR allocation, hardware-guarded loads, or explicit scheduling, which contribute to this gap. AITER is near parity with hipBLASLt at $M = N = K = 1024$, but its normalized throughput decreases at larger sizes as Split K becomes less effective than grid partitioning.

Flash Attention. Ave matches AITER's handwritten assembly. Atrex reaches 0.91× through the contaminated retrieval [1]. Among uncontaminated agentic baselines, Ave's normalized throughput is 6.6× and 1176× relative to KSearch and CUDAForge, respectively, and 3× relative to KernelFalcon. KSearch and CUDAForge omit matrix cores for $\mathbf{QK}^T$ and $\mathbf{PV}$. KernelFalcon's Triton kernel neither reorganizes and stages $\mathbf{V}$ for an on-the-fly transpose nor pipelines and interleaves instructions. The evaluated ROCm Triton baseline cannot express wavefront specialization or fine-grained pipeline scheduling. When paired with DSv4F, Atrex also spends substantial compilation time in a trial-and-error search for a working compiler configuration.

Fused MoE. Ave and AITER partition tokens and experts, use matrix cores, and quantize up-projection intermediates to FP8, enabling FP8 matrix cores with 16× the throughput of scalar units. Ave's normalized throughput is 8× relative to KernelFalcon and 51× relative to CUDAForge. A correct fused MoE kernel must preserve blockwise quantization, route tokens through sorted expert maps, and group accesses for coalescing. With DeepSeek V4 Pro, Atrex and KSearch fail to produce stable kernels.

KernelFalcon uses one stage with DSv4 and two with GPT and DSv4F. Successful KSearch and CUDAForge kernels also fuse the up projection and activation before writing a workspace. These two-stage designs omit FP8 requantization before the down projection and therefore cannot use FP8 matrix cores. They also omit expert parallelism and full vectorization despite 16-byte alignment.

Table 1 shows that Ave remains effective across models, whereas competing frameworks are more model-sensitive. GPT usually produces higher-throughput kernels but can generate excessive code, causing instruction-cache thrashing in CUDAForge attention. DSv4 and DSv4F more often omit optimizations or choose ineffective strategies. For example, DSv4 gives KernelFalcon a low-occupancy, single-stage MoE kernel. Attention invariant checks take only 10–40 ms, negligible beside LLM inference.

9.2 Ablation Study of Ave's Optimization

To answer Q2, we use GPT and the same five attention configurations as Q1. We cumulatively apply five optimizations to naïve flash attention: on-demand $\mathbf{V}^\mathsf{T}$ materialization, asynchronous loads of $\mathbf{Q}$ and $\mathbf{K}$ (+Async), bank-conflict mitigation (+Bank), pipelining with wavefront specialization (+Pipe.), and instruction scheduling (+Schedule). Ave also exposes fine-grained register allocation and data-flow invariants.

Figure 3(a) shows throughput increasing monotonically across all five stages. +Pipe gives the largest incremental gain: 75–116% across sequence lengths and 100% by geometric mean. The final configuration reaches 2.7–3.6× Naïve throughput. Effective TFLOPS use dense-attention FLOPs, including the causally skipped upper triangle. Figure 3(b) isolates harness components while retaining unit tests and runtime feedback in every configuration. Because each knowledge base (KB) entry normally pairs an optimization recipe with invariant templates, the no-invariant ablation uses the same KB after stripping those templates. In (i), feedback prompts +Async and +Bank but cannot diagnose their ineffective implementations, leaving performance near Naïve. In (ii), the KB provides optimization recipes and invariant

[1] Atrex copies the AITER reference kernel from its knowledge base rather than deriving it through optimization.

	Syntax reference only					In-context examples					In-context optimization					
Dataset–Model	Pass@1/3	Turns	Tok.	Cost	Time	Pass@1/3	Turns	Tok.	Cost	Time	>1×	Mean/Min/Max	Turns	Tok.	Cost	Time
L1-GPT	96/99	19.20	0.81	$0.85	156/18/3	98/100	16.71	0.63	$0.73	158/15/6	54	1.10/0.12/25.56	4.11	0.10	$0.67	319/42/5
L1-DSv4	94/96	22.45	1.21	$0.13	190/28/14	95/95	19.69	0.76	$0.09	226/25/35	40	0.64/0.01/29.23	4.18	0.14	$0.18	782/41/18
L1-DSv4F	97/100	17.71	0.94	$0.05	137/17/4	100/100	15.64	0.70	$0.04	97/14/6	50	0.99/0.09/30.45	4.52	0.25	$0.16	1481/43/5
L2-GPT	82/87	37.59	1.96	$1.81	289/41/4	84/88	34.30	1.94	$1.78	312/38/18	29	1.03/0.07/19.80	5.62	0.20	$1.46	605/54/11
L2-DSv4	82/86	41.60	3.25	$0.30	287/58/23	83/86	30.78	1.82	$0.19	258/35/61	18	0.47/0.03/6.52	4.92	0.17	$0.24	968/39/14
L2-DSv4F	77/86	40.07	3.79	$0.13	281/24/14	79/84	30.31	2.63	$0.10	170/20/13	34	0.99/0.05/41.66	5.59	0.34	$0.23	2043/44/10

Table 3. KernelBench generation under syntax-reference and in-context prompting, followed by optimization of the in-context results.

Dataset–Model	Success	Turns	Tok.	Cost	Time
No invariants					
$\widetilde{\text{L1}}$-GPT	8	7.00	0.24	$1.74	607/45/1
$\widetilde{\text{L1}}$-DSv4	1	4.00	0.19	$0.28	1105/40/2
$\widetilde{\text{L1}}$-DSv4F	1	12.00	0.86	$0.58	4968/99/6
$\widetilde{\text{L2}}$-GPT	12	6.50	0.25	$1.73	719/60/2
$\widetilde{\text{L2}}$-DSv4	7	6.43	0.28	$0.44	1763/71/2
$\widetilde{\text{L2}}$-DSv4F	7	6.57	0.37	$0.23	1963/67/10
With invariants					
$\widetilde{\text{L1}}$-GPT	18	6.06	0.22	$1.54	679/40/2
$\widetilde{\text{L1}}$-DSv4	16	5.88	0.26	$0.43	1956/38/2
$\widetilde{\text{L1}}$-DSv4F	19	6.47	0.39	$0.26	2313/43/2
$\widetilde{\text{L2}}$-GPT	25	6.12	0.22	$1.61	656/56/6
$\widetilde{\text{L2}}$-DSv4	26	5.54	0.22	$0.34	1273/43/4
$\widetilde{\text{L2}}$-DSv4F	23	5.74	0.37	$0.26	2100/50/4

Table 4. Effect of data-flow invariants on optimizing 60 selected KernelBench tasks.

templates, but without ICRL, the planner attempts +Schedule before +Pipe. Scheduling has no effect, leaving performance at +Bank. In (iii), ICRL orders the steps, but without invariants the agent cannot diagnose and repair a $\mathbf{V}^{\mathsf{T}}$ layout that mispairs reduction coordinates, so performance remains near Naïve. The full system in (iv) completes the sequence: $\mathbf{V}^{\mathsf{T}}$, +Async, +Bank, +Pipe., and +Schedule.

9.3 Generality Across Architectures and Kernels

To answer *Q3*, we evaluate both cross-architecture portability and generality beyond the production kernels in §9.1. For portability, we add an NVIDIA Hopper backend and use GPT to optimize the same five configurations of each workload on H200. Against cuBLASLt [51], vLLM's FlashAttention-3 fork (b3964b1) [68], and vLLM's DeepGEMM fork (a6b593d) [21], Ave achieves geometric-mean normalized throughputs of 0.98×, 1.00×, and 0.99× over the five GEMM, attention, and fused-MoE configurations, respectively.

To assess cross-kernel generality, we evaluate 100 Level 1 (L1) and 100 Level 2 (L2) KernelBench tasks on MI300X with each model. Because the Ave DSL is absent from their training corpora, Table 3 compares prompting with only a compact syntax reference against prompting with in-context examples. Generation uses only unit-test feedback and stops at the first test-passing candidate. Among generations using in-context examples, manual review flags direct PyTorch calls or input-specific shortcuts in 3 L1-DSv4, 6 L2-GPT, 13 L2-DSv4, and 5 L2-DSv4F cases. Optimization starts from this candidate and runs only for the in-context condition. One L2-DSv4 optimization ignores instructions and falls back to PyTorch. The reported pass rates count these outputs as failures. In-context examples improve Pass@1 while reducing turns, tokens, and API cost.

During optimization, Ave applies layout transformations, fusion, vectorized loads, shared-memory staging, and instruction scheduling. Some kernels remain below parity because their references call hipBLASLt or MIOpen. DSv4 and DSv4F have lower aggregate pass rates and mean throughput than GPT, partly because they more often omit transformations such as im2col [12].

9.4 Effectiveness and Cost of Data-Flow Invariants

To isolate the impact of data-flow invariants (*Q4*), we compare the full system with an ablation that removes invariant templates and, consequently, generates no tag functions, assertions, or invariant-violation feedback. We select KernelBench GEMM and convolution tasks from L1 ($\widetilde{\text{L1}}$) and L2 ($\widetilde{\text{L2}}$). Each task requests MFMA matrix multiplication, software pipelining, and HW OOB-guarded loads. Success requires applying all three.

Table 4 shows that invariants improve success in every dataset–model pair. They generally reduce turns, tokens, and API cost, while their effect on wall time is mixed. Thus, invariants consistently make intrusive transformations more reliable.

10 Related Work

Synthesizing GPU Kernels with LLMs. Frontier coding models [23, 55] combined with agentic workflows [6, 9, 15, 18, 25, 26, 30, 32, 56, 70, 72, 76, 78] have made generating GPU

kernels from natural language specifications practical. These workflows differ along three axes: reward signals, optimization knowledge, and execution quality. Most use correctness and relative performance gains as rewards, which are susceptible to reward hacking [39]. The concurrent μCUTLASS system instead uses Speed-of-Light guidance, penalizing claims that exceed hardware capabilities to reduce reward hacking [30]. Sources include fine-tuning and reinforcement learning [64], world models [9], and evolutionary search [43].

Reliable execution of complex optimizations is essential for meaningful performance gains on production kernels. Ave incorporates data-flow invariants as dense process rewards that guide optimization. Exposing low-level hardware and compiler intrinsics further enables optimizations such as instruction scheduling, distinguishing Ave from systems limited to end-to-end correctness and performance rewards.

ICL and ICRL. In-context learning (ICL) enables LLMs to adapt to new downstream tasks from examples provided in the prompt without model parameter updates [49]. Furthermore, in-context reinforcement learning (ICRL) extends ICL by treating the prompt as a dynamic learnable policy and updating it based on reward signals. For example, TextGrad [74] provides a general framework for computing text-based gradients over LLM outputs. In terms of kernel code generation, KernelBlaster [26] applies ICRL to CUDA kernel optimization, maintaining a memory of past optimization trajectories and using them to improve cross-task generalization. Ave builds on these techniques: we use ICRL to learn planner prompts that bind optimization knowledge to concrete kernels, with data-flow invariant violations serving as process rewards alongside runtime performance.

DSL Designs for GPU Kernels. GPU DSLs balance productivity against low-level control through their choice of abstraction. Triton describes workloads at the block level, hiding explicit memory layouts and synchronization. Halide [59] and Exo [35] separate algorithm specification from scheduling policy for tensor workloads. Tile-based DSLs [24, 30, 52, 69] further structure programs around tiles as primitives for data movement and matrix multiplication, shifting implementation complexity to compilers. This convenience can impose an abstraction tax. Triton extensions such as Gluon [67] recover performance by exposing direct control over memory layouts, warp specialization, register allocation, and instruction scheduling.

The Ave DSL combines tile abstractions with explicit compiler controls for LLM-driven development. Its restricted memory model keeps static analysis tractable, while compile-time data-flow invariants provide dense structural feedback during optimization. Its Triton-like syntax lets LLMs reuse knowledge from training.

Optimizing Compilers for Tensor Programs. Tensor compilers [14, 29, 42, 46, 61, 71, 79, 80] lower DSLs and tensor programs to GPU binaries. Their optimizations target high tensor core utilization, efficient memory layout, software pipelining, operator fusion, and overlapping computation with memory access. They rely on heuristics and exhaustive search to find efficient schedules. Note that Ave complements these compilers: we expose the hardware SIMT programming model along with low-level intrinsics in the DSL, and delegate schedule exploration to LLMs.

Invariants and Information Flow Systems. Invariants provide a concise way to specify and reason about end-to-end properties across multi-layered systems [47, 65, 73]. Information flow analysis enforces such invariants at the system level. HiStar [75] assigns security labels to data, defines a partial order on labels, tracks their propagation, and permits information flow only along that order. JFlow [50] augments Java with information labels and enforces information flows at the language level. These prior systems focus on enforcing security policies. Ave adapts the same tracking and enforcement mechanisms to GPU kernel optimization, using tag propagation to check correct operand pairings and giving LLMs a cross-layer view of relevant data-flow constraints.

Formal verification [11, 13, 37, 63] can detect invariant violations precisely, but multiplies development effort by 2–6× [37] for sequential programs. Verifying concurrent GPU kernels demands additional modeling effort [7]. Ave instead adopts lightweight static analysis to report actionable validation failures, enabling the agentic loop to iterate automatically. The current prototype uses Z3 [20] to reason about layout algebra. A refined version could leverage polyhedral analysis [8, 28, 57] for faster validation.

11 Conclusion

We introduced Ave, an agentic framework that uses compile-time data-flow invariants to make LLM-generated GPU optimizations more reliable. Its tile-based DSL and static analysis turn invariant violations into concrete witnesses, guiding agents through intrusive transformations without runtime overhead. With ICRL, Ave reaches near-library performance on AMD MI300X and generalizes to NVIDIA H200. More broadly, pairing domain abstractions with actionable correctness feedback offers a path to reliable agentic optimization wherever correctness depends on cross-layer data flow.

Acknowledgments

We thank our shepherd, the anonymous reviewers, and Mark Hill for their thoughtful feedback. We also thank InnoMatrix for providing the evaluation servers. This work was partially supported in part by donation grant PD26EG02 to CSE HKUST, HKUST startup grant R9895, RGC-ECS project 26218024, RGC-NSFC project CRS_HKUST601/24, and National Natural Science Foundation of China grant 62232015.

References

[1] AMD. AITER: AI Tensor Engine for ROCm. https://github.com/ROCm/aiter.

[2] AMD. AMD Instinct MI300 Instruction Set Architecture. https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/amd-instinct-mi300-cdna3-instruction-set-architecture.pdf.

[3] AMD. AMD Instinct MI300X workload optimization. https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/inference-optimization/workload.html.

[4] AMD. hipBLASLt: General Matrix-Matrix Operations Library for AMD GPUs. https://rocm.docs.amd.com/projects/hipBLASLt/en/latest.

[5] AMD. ROCm Software. https://github.com/ROCm/ROCm.

[6] Carlo Baronio, Pietro Marsella, Ben Pan, et al. Kevin: Multi-Turn RL for Generating CUDA Kernels *(ICLR'26)*.

[7] Adam Betts, Nathan Chong, Alastair Donaldson, et al. GPUVerify: a verifier for GPU kernels *(OOPSLA '12)*.

[8] Uday Bondhugula, Albert Hartono, J. Ramanujam, et al. A practical automatic polyhedral parallelizer and locality optimizer *(PLDI '08)*.

[9] Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, et al. K-Search: LLM Kernel Generation via Co-Evolving Intrinsic World Model. arXiv:2602.19128.

[10] Jack Carlisle, Jay Shah, Reuben Stern, et al. Categorical Foundations for CuTe Layouts. arXiv:2601.05972.

[11] Bodhisatwa Chatterjee, Drew Zagieboylo, Sana Damani, et al. ProofWright: Towards Agentic Formal Verification of CUDA. arXiv:2511.12294.

[12] Kumar Chellapilla, Sidd Puri, and Patrice Simard. High Performance Convolutional Neural Networks for Document Processing *(IWFHR'06)*.

[13] Haogang Chen, Daniel Ziegler, Tej Chajed, et al. Using Crash Hoare Logic for Certifying the FSCQ File System *(USENIX ATC'16)*.

[14] Tianqi Chen, Thierry Moreau, Ziheng Jiang, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning *(OSDI'18)*.

[15] Terry Chen, Zhifan Ye, Bing Xu, et al. AVO: Agentic Variation Operators for Autonomous Evolutionary Search. arXiv:2603.24517.

[16] Xinhao Cheng, Zhihao Zhang, Yu Zhou, et al. MPK: A Compiler and Runtime for Mega-Kernelizing Tensor Programs *(OSDI'26)*.

[17] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints *(POPL'77)*.

[18] Weinan Dai, Hanlin Wu, Qiying Yu, et al. CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation. arXiv:2602.24286.

[19] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. arXiv:2307.08691.

[20] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver *(TACAS'08)*.

[21] DeepSeek-AI. DeepGEMM: A High-Performance Tensor Core Kernel Library. https://github.com/deepseek-ai/DeepGEMM.

[22] DeepSeek-AI. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. arXiv:2512.02556.

[23] DeepSeek-AI, Anyi Xu, Bangcai Lin, et al. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348.

[24] Yaoyao Ding, Bohan Hou, Xiao Zhang, et al. Tilus: A Tile-Level GPGPU Programming Language for Low-Precision Computation *(ASPLOS '26)*.

[25] Juncheng Dong, Yang Yang, Tao Liu, et al. STARK: Strategic Team of Agents for Refining Kernels *(ICLR'26)*.

[26] Shengjun Kris Dong, Sahil Modi, Dima Nikiforov, et al. KernelBlaster: Continual Cross-Task CUDA Optimization via Memory-Augmented In-Context Reinforcement Learning. arXiv:2602.14293.

[27] GLM-5-Team. GLM-5: from Vibe Coding to Agentic Engineering. arXiv:2602.15763.

[28] Tobias Grosser, Armin Groesslinger, and Christian Lengauer. Polly - Performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters* 22, 4 (2012), 1250010.

[29] Yue Guan, Xinwei Qiang, Zaifeng Pan, et al. Mercury: Unlocking Multi-GPU Operator Optimization for LLMs via Remote Memory Scheduling *(SOSP '25)*.

[30] Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, et al. Improving Efficiency of GPU Kernel Optimization Agents using a Domain-Specific Language and Speed-of-Light Guidance. arXiv:2603.29010.

[31] Chris Hawblitzel, Jon Howell, Manos Kapritsos, et al. IronFleet: proving practical distributed systems correct *(SOSP '15)*.

[32] Bodun Hu, Yoga Sri Varshan V, Saurabh Agarwal, et al. CUCo: An Agentic Framework for Compute and Communication Co-design. arXiv:2603.02376.

[33] William Hu, Drew Wadsworth, Sean Siddens, et al. HipKittens: Fast and Furious AMD Kernels. arXiv:2511.08083.

[34] Rodrigo Huerta, Mojtaba Abaie Shoushtary, José-Lorenzo Cruz, et al. Dissecting and Modeling the Architecture of Modern GPU Cores *(MICRO '25)*.

[35] Yuka Ikarashi, Kevin Qian, Samir Droubi, et al. Exo 2: Growing a Scheduling Language *(ASPLOS '25)*.

[36] KernelWiki. KernelWiki — Blackwell & Hopper Kernel Optimization Knowledge Base. https://github.com/mit-han-lab/KernelWiki.

[37] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, et al. seL4: formal verification of an OS kernel *(SOSP '09)*.

[38] Monica Lam. Software pipelining: an effective scheduling technique for VLIW machines *(PLDI '88)*.

[39] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, et al. Towards Robust Agentic CUDA Kernel Benchmarking, Verification, and Optimization. arXiv:2509.14279.

[40] Chris Lattner, Mehdi Amini, Uday Bondhugula, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation *(CGO'21)*.

[41] Felix Li, Shijie Feng, Carlus Huang, et al. FlyDSL. https://github.com/ROCm/FlyDSL.

[42] Yijin Li, Jiacheng Zhao, Sun Qianqi, et al. SIRIUS: Harvesting Whole-Program Optimization Opportunities for DNNs *(MLSys'23)*.

[43] Gang Liao, Hongsen Qin, Ying Wang, et al. KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta. arXiv:2512.23236.

[44] Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the Middle: How Language Models Use Long Contexts *(TACL'24)*.

[45] Weile Luo, Ruibo Fan, Zeyu Li, et al. Benchmarking and Dissecting the Nvidia Hopper GPU Architecture *(IPDPS'24)*.

[46] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, et al. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks *(OSDI'20)*.

[47] Haohui Mai, Edgar Pek, Hui Xue, et al. Verifying security invariants in ExpressOS *(ASPLOS '13)*.

[48] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. arXiv:1805.02867.

[49] Giovanni Monea, Antoine Bosselut, Kianté Brantley, et al. LLMs Are In-Context Bandit Reinforcement Learners *(COLM'25)*.

[50] Andrew C. Myers. JFlow: practical mostly-static information flow control *(POPL '99)*.

[51] NVIDIA. cuBLAS Library Documentation. https://docs.nvidia.com/cuda/archive/12.8.0/cublas.

[52] NVIDIA. CUTLASS: CUDA Templates for Linear Algebra Subroutines and Solvers. https://github.com/NVIDIA/cutlass.

[53] NVIDIA. Parallel Thread Execution ISA Version 9.3. https://docs.nvidia.com/cuda/parallel-thread-execution.

[54] OpenAI. Lightweight coding agent that runs in your terminal. https://github.com/openai/codex.

[55] OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. https://openai.com/index/gpt-5-6.

[56] Anne Ouyang, Simon Guo, Simran Arora, et al. KernelBench: Can LLMs Write Efficient GPU Kernels? *(ICML'25)*.

[57] Benoît Pradelle, Benoît Meister, Muthu Baskaran, et al. Polyhedral Optimization of TensorFlow Computation Graphs *(ProTools'18)*.

[58] Fernando Magno Quintão Pereira and Jens Palsberg. Register allocation by puzzle solving *(PLDI '08)*.

[59] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, et al. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines *(PLDI '13)*.

[60] G. Ramalingam. The undecidability of aliasing. *ACM Trans. Program. Lang. Syst.* 16, 5 (Sept. 1994), 1467–1471.

[61] Amit Sabne. XLA: Compiling Machine Learning for Peak Performance *(DEEM'20)*.

[62] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer *(ICLR'17)*.

[63] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, et al. Push-Button Verification of File Systems via Crash Refinement *(OSDI'16)*.

[64] Songqiao Su, Xiaofei Sun, Xiaoya Li, et al. CUDA-L2: Surpassing cuBLAS Performance for Matrix Multiplication through Reinforcement Learning. arXiv:2512.02551.

[65] Shuo Tang, Haohui Mai, and Samuel T. King. Trust and protection in the Illinois browser operating system *(OSDI'10)*.

[66] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations *(MAPL'19)*.

[67] Triton. Gluon. https://github.com/triton-lang/triton/tree/main/lib/Dialect/Gluon.

[68] vllm-project. Fast and memory-efficient exact attention. https://github.com/vllm-project/flash-attention.

[69] Lei Wang, Yu Cheng, Yining Shi, et al. TileLang: Bridge Programmability and Performance in Modern Neural Kernels *(ICLR'26)*.

[70] Laura Wang and PyTorch Team. KernelFalcon: Autonomous GPU Kernel Generation via Deep Agents. https://pytorch.org/blog/kernelfalcon-autonomous-gpu-kernel-generation-via-deep-agents.

[71] Mengdi Wu, Xinhao Cheng, Shengyu Liu, et al. Mirage: a multi-level superoptimizer for tensor programs *(OSDI '25)*.

[72] Lingyun Yang, Yuxiao Wang, Shenghao Liang, et al. Are LLM-Generated GPU Kernels Production-Ready? A Trace-Driven Benchmark and Optimization Agent. arXiv:2607.14541.

[73] Alexander Yip, Xi Wang, Nickolai Zeldovich, et al. Improving application security with data flow assertions *(SOSP '09)*.

[74] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, et al. TextGrad: Automatic "Differentiation" via Text. arXiv:2406.07496.

[75] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, et al. Making Information Flow Explicit in HiStar *(OSDI'06)*.

[76] Yi Zhai, Sijia Yang, Keyu Pan, et al. Enabling Tensor Language Model to Assist in Generating High-Performance Tensor Programs for Deep Learning *(OSDI'24)*.

[77] Shulai Zhang, Ningxin Zheng, Haibin Lin, et al. COMET: Fine-grained Computation-communication Overlapping for Mixture-of-Experts *(MLSys'25)*.

[78] Zijian Zhang, Rong Wang, Shiyang Li, et al. CudaForge: An Agent Framework with Hardware Feedback for CUDA Kernel Optimization. arXiv:2511.01884.

[79] Jie Zhao, Bojie Li, Wang Nie, et al. AKG: automatic kernel generation for neural processing units using polyhedral transformations *(PLDI'21)*.

[80] Lianmin Zheng, Chengfan Jia, Minmin Sun, et al. Ansor: Generating High-Performance Tensor Programs for Deep Learning *(OSDI'20)*.